

Console I/O and Compilation

CSC 230 : C and Software Tools

NC State Department of Computer
Science

Topics for Today

- Termination and exit status
- Terminal Input and Output
 - Character I/O
 - Formatted I/O
- Meet the Preprocessor
- Tokenization
- Coding Style

Constants for Exit Status

- You already know:
 - main() has a return type of int
 - indicating program success (return 0) or failure (return non-zero).
- There are named constants to make this more readable.

```
return EXIT_SUCCESS;
```

```
return EXIT_FAILURE;
```

- These are both defined in ***stdlib.h***
- Include `stdlib.h` near the top of your program to get these constants.

```
#include <stdlib.h>
```

Using the Standard Library

- In C, I/O is provided by functions in the *standard library*
 - This library is expected on all platforms
 - Including *header files* gives you access to parts of the library.
- Some of the most general language features are in standard-lib.

```
#include <stdlib.h>
```

This is a header file.

- To perform Input and Output, you'll need the standard-I/O header:

```
#include <stdio.h>
```

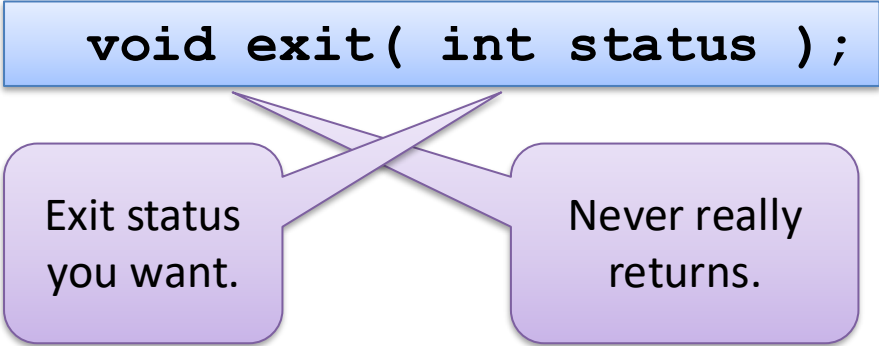
- These #include lines are *preprocessor directives*, telling the compiler to read these files first in order to understand the rest of the source code.

Anything starting with a # is a preprocessor directive.

Exiting from Anywhere

- Inside main, you can use return to exit.
- ... but, this won't work from other functions
- For that, we can use the exit function.

```
void exit( int status );
```



Exit status
you want.

The diagram shows a blue box containing the function signature `void exit( int status );`. Two purple callout boxes are positioned below it. The left box contains the text 'Exit status you want.' and the right box contains 'Never really returns.'. Two lines originate from the 'status' parameter in the function signature, one pointing to the left box and one pointing to the right box. The lines cross each other in the center.

Never really
returns.

- Also declared in the **stdlib.h** header.

Streams

- A *stream* is a file or device we can read or write
- Just like in Java, a C program starts with three streams it can use
 - *Standard input* (input typed at the terminal)
 - *Standard output* (output to the terminal)
 - *Standard error* (more output to the terminal)
- To a program, reading and writing to the terminal looks just like reading or writing a file
 - We can even signal the End-Of-File condition on standard input:
 - Type CTRL-D if your program is running in Linux or MacOS (or logged in to a Linux machine from Windows)
 - Or, CTRL-Z if your program is running in Windows.

Redirecting Standard Streams

- We can redirect these streams to or from actual files (without the program even noticing)
 - We won't learn about file I/O for a while, but this will let us get by without it.
- From the terminal, you can redirect standard input from a file

```
$ myProgram < input_file.txt
```

- ... or standard output to a file.

```
$ myProgram > output_file.txt
```

Reading just one Character

- `stdio.h` provides a function
`int getchar( void )`

It returns an int, a numeric code for the next character read.

It doesn't take any parameters.

- Returns a code for the next character read (as an integer from 0 .. 255)
- ... or, if there's no more input, the constant *EOF* (the value -1)
- That's why its return type is int instead of char (students seem to forget this)

Remember

getchar() returns **int**

- Every character has a code that will fit in a char variable.
- .. but not the EOF value.
 - If you try to store it in a char, you may think you've reached EOF when you haven't.

Character Representation

- A **char** variable can hold a small integer
 - Just 256 different values, what you can fit in 8 bits (a *byte*).
 - (the standard doesn't absolutely guarantee char is a byte ... but, it's pretty much always a byte).
- Each symbol is as a represented by a numeric code.
- **ASCII** is one standard for what numbers go with each symbol.
 - **ASCII** = **A**merican **S**tandard **C**ode for **I**nformation **I**nterchange
 - It's a rule giving the code for common, western characters.
- The C standard doesn't require ASCII
 - But that's what you'll get on the common platform.

ASCII

0	nul	1	soh	2	stx	3	etx	4	eot	5	enq	6	ack	7	bel
8	bs	9	ht	10	nl	11	vt	12	np	13	cr	14	so	15	si
16	dle	17	dc1	18	dc2	19	dc3	20	dc4	21	nak	22	syn	23	etb
24	can	25	em	26	sub	27	esc	28	fs	29	gs	30	rs	31	us
32	sp	33	!	34	"	35	#	36	\$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124		125	}	126	~	127	del

Numeric Code

Character
(or its name)

ASCII

0	nul	1	soh	2	stx	3	etx	4	eot	5	enq	6	ack	7	bel
8	bs	9	ht	10	nl	11	vt	12	np	13	cr	14	so	15	si
16	dle	17	dc1	18	dc2	19	dc3	20	dc4	21	nak	22	syn	23	etb
24	can	25	em	26	sub	27	esc	28	fs	29	gs	30	rs	31	us
32	sp	33	!	34	"	35	#	36	\$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124		125	}	126	~	127	del

- You don't need to memorize this table
- ... but, there are a few things to notice and remember:
 - Letters and digits are consecutive and in order (good)
 - Capital and lower-case letters are in different parts of the table (with a little gap in between)
 - Some (low-numbered) codes are for control characters

ASCII

0	nul	1	soh	2	stx	3	etx	4	eot	5	enq	6	ack	7	bel
8	bs	9	ht	10	nl	11	vt	12	np	13	cr	14	so	15	si
16	dle	17	dc1	18	dc2	19	dc3	20	dc4	21	nak	22	syn	23	etb
24	can	25	em	26	sub	27	esc	28	fs	29	gs	30	rs	31	us
32	sp	33	!	34	"	35	#	36	\$	37	%	38	&	39	'
40	(	41	)	42	*	43	+	44	,	45	-	46	.	47	/
48	0	49	1	50	2	51	3	52	4	53	5	54	6	55	7
56	8	57	9	58	:	59	;	60	<	61	=	62	>	63	?
64	@	65	A	66	B	67	C	68	D	69	E	70	F	71	G
72	H	73	I	74	J	75	K	76	L	77	M	78	N	79	O
80	P	81	Q	82	R	83	S	84	T	85	U	86	V	87	W
88	X	89	Y	90	Z	91	[	92	\	93	]	94	^	95	_
96	`	97	a	98	b	99	c	100	d	101	e	102	f	103	g
104	h	105	i	106	j	107	k	108	l	109	m	110	n	111	o
112	p	113	q	114	r	115	s	116	t	117	u	118	v	119	w
120	x	121	y	122	z	123	{	124		125	}	126	~	127	del

- Also notice:
 - A byte could store 256 different codes, but this table has half that.
 - Codes 128 – 255 may be application/platform dependent.
 - The symbol '0' isn't represented by the code zero ... that code has a special job in C.

Character Codes

- It's usually a bad idea to use numbers directly for ASCII codes.

```
char ch = 97;
```

- That's what a *literal character* is for ... a symbol inside single quotes.
- This is much easier for most common symbols:

```
char ch1 = 'a';  
char ch2 = '7';  
char ch3 = '$';
```

- Java works the same way ... but it uses Unicode rather than plain-old ASCII.
- How about codes for characters you can't type?

Special Characters

- Some character must be entered with *escape sequences*:

Escape Sequence	Character
\0	A null (more on this later)
\'	Single quote
\"	Double quote
\\	Backslash
\n	Newline
\t	Horizontal tab

Computing with Characters

- The ordering of ASCII codes makes some things easy.
- Lower-case letters are consecutive.
 - A character is a lower-case letter if it's in the right range.

```
int ch = getchar();  
if ( ch >= 'a' && ch <= 'z' )  
    ...
```

- The same technique works for digits or capital letters.

```
int ch = getchar();  
if ( ch >= '0' && ch <= '9' )  
    ...
```

Computing with Characters

- If **ch** stores the code for a decimal digit:

```
int ch = getchar();  
if ( ch >= '0' && ch <= '9' )  
    val = ch - '0';
```

I'm the value of
the digit.

I'm a character
code.

- The same works for letters:

```
int ch = getchar();  
if ( ch >= 'a' && ch <= 'z' )  
    cap = ch - 'a' + 'A';
```

I'm a character code
for a capital letter.

I'm a character code
for a lower-case letter.

Working with Characters

- C provides some common character operations.
- The **ctype.h** header declares many functions for working with characters.

```
#include <ctype.h>
```

- You can check if a character is a letter.

```
bool isalpha( int ch );
```

Character to check.

True if ch is a letter.

- You can check if a character is whitespace.

```
bool isspace( int ch );
```

These functions don't really return Boolean ...
we're pretending they do for now.

Working with Characters

- Check if a character is a digit.

```
bool isdigit( int ch );
```

- Check if a character is printable (not a control character)

```
bool isprint( int ch );
```

- There are lots of others (but you just need to know the first four)

```
bool isalnum( int ch );  
bool ispunct( int ch );  
bool isupper( int ch );  
...
```

Working with Characters

- There's a function to convert to upper-case:

```
int toupper( int ch );
```

Original character goes in here.

Upper-case version comes out here (for letters, same as input for everything else).

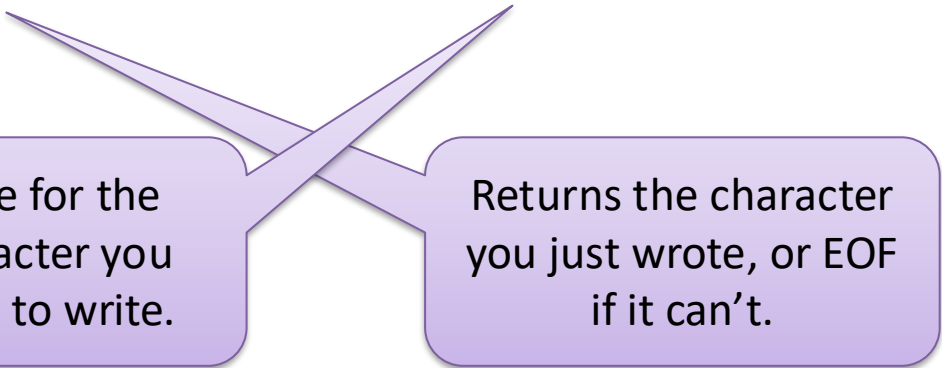
- There's a similar function for going the other way:

```
int tolower( int ch );
```

Writing just one Character

- `stdio.h` also provides a function for sending just one character to the terminal:

```
int putchar( int c )
```



Code for the character you want to write.

Returns the character you just wrote, or EOF if it can't.

Character I/O Example

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>
```

```
int main()
```

```
{
```

```
    int ch = getchar();
```

```
    while ( ch != EOF ) {
```

```
        if ( ch == ' ' )
```

```
            putchar( '-' );
```

```
        else
```

```
            putchar( toupper( ch ) );
```

```
        ch = getchar();
```

```
    }
```

```
    return EXIT_SUCCESS;
```

```
}
```

Read just one character.

See if we succeeded.

Print just one character.

Use the named constant for success.

Formatted Output

- The printf() function is good for generating formatted output
- You probably saw a similar function in Java. It works like:

```
printf( "value: %6.2f\n", 3.1415926 );
```

This is a *format string*.
Most of it gets printed
literally.

... but not parts like this.
This is a *conversion
specifier*.

Each conversion specifier
says how to print one of
the remaining
parameters.

Making Sense of Conversion Specifiers

The percent says “Here comes a conversion specifier”

This last character (or characters) says what to print, here it’s a double.

`%6.2f`

Minimum *Field Width*
This part is optional. It says, “use **at least** 6 characters of output”.

Right-aligned by default,
Negative for left-aligned.

Precision
This part is optional. For real numbers, it says “round output to two fractional digits”.

Conversion Specifier Examples

- There are lots of ways to print a value like 33.3:

Conversion Specifier	Output
%7.1f	33.3
%-14.8f	33.30000000
%10.20f	33.299999999999999715783

Plenty of space, 3 spaces on the left.

Three extra spaces on this end, to get to 14.

Already 23 characters wide; no extra space.

Conversion Specifier Overview

- printf() can output lots of different types of values.
- Here are some we'll need for now.

Conversion Specifier	Output
%c	A single character (given its numeric character code)
%d	A decimal integer
%ld	A long integer
%f	A float or double
%s	A string (later)

Reading Formatted Input

- **printf()** is for creating formatted output to the terminal.
- Its friend, **scanf()**, is for reading formatted input.
- The **scanf()** function converts sequences of characters into other types
 - Like int, double, string, ...
 - It can skip whitespace and other input you tell it to ignore
- It's a lot like **printf()**

```
int scanf( const char *fmt, more...args )
```

- It takes a format string, followed by a variable number of additional arguments
- It matches characters from the input and assigns them to the subsequent arguments.

Using scanf()

- The format string can contain one or more *conversion specifiers*.
- Each of these fills in the value of one of the remaining arguments.
- You can't do this with *pass-by-value*

```
scanf( "%d", val );
```

This couldn't possibly work ... a function can't change what's in this variable.

- So, C uses a mechanism for *pass-by-reference*

```
scanf( "%d", &val );
```

With the &, we're passing val so scanf can change what's in it.

Using Scanf

- scanf() can match multiple conversion specifiers.

```
scanf( "%f%d", &floatVal, &intVal );
```

- It returns the number of specifiers matched

```
int matches = scanf( "%f%d", &floatVal, &intVal );
```

- Fewer on bad input
- ... or EOF if it reaches the end-of-file before matching any.
- And, it leaves a parameter unchanged if it can't match its conversion specifier.

Using Scanf

- The return value from scanf() is important.
- You could use it like this:

```
printf( "Enter an integer or two: " );

int v1, v2, v3;
int matches = scanf( "%d%d%d", &v1, &v2, &v3 );
if ( matches == 1 ) {
    printf( "You entered %d\n", v1 );
} else if ( matches == 2 ) {
    printf( "You entered %d and %d\n", v1, v2 );
} else if ( matches == 3 ) {
    printf( "You entered more than two numbers\n" );
} else if ( matches == 0 ) {
    printf( "That's not a number\n" );
} else {
    printf( "There wasn't any input.\n" );
}
```

Conversion Specifier Zoo

- scanf() understands almost exactly the same conversion specifier as printf()
- ... but, there are some differences ☹️

Parse input as	Conversion Specifier
int	%d (in decimal)
float	%f
double	%lf
long int	%ld
A single character	%c
A whole string	%s

Matches the longest sequence of characters that looks like an int.

... longest sequence of characters that looks like a real number.

This is a little different from printf().

Most of these skip leading whitespace ... but not this one.

Fun with Scanf

- Consider the code:

```
float fval = 1.0f;  
int ival = 2;  
int matches = scanf( "%f%d", &fval, &ival );
```

- What will happen for the following input?

1.23 4

- Or this input

3 4.12

- ... or this input

37.22b5

Formatted I/O Example

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int val;

    while ( scanf( "%d", &val ) == 1 ) {
        printf( "%6d\n", val );
    }

    return EXIT_SUCCESS;
}
```

While we keep matching
one decimal integer.

Print it in a 6-character
field.

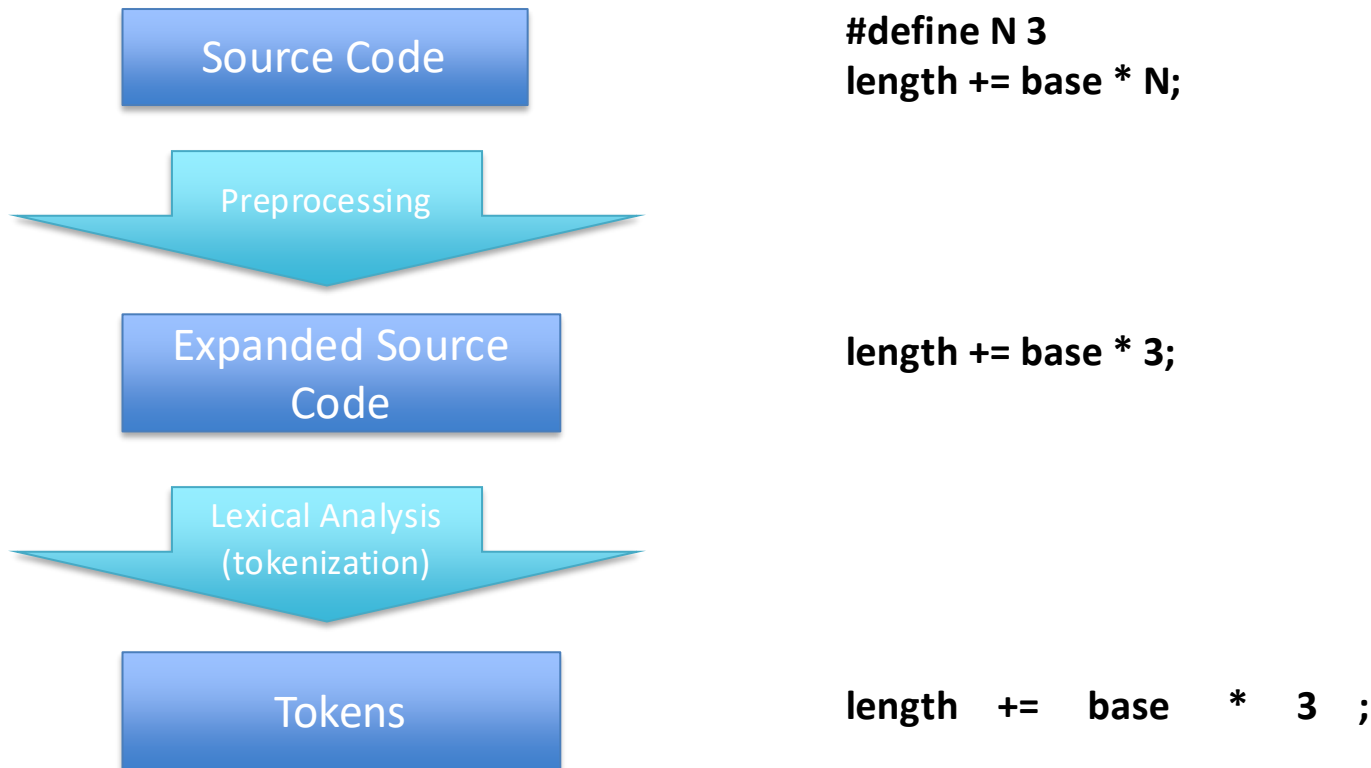
```
25
-12  77
green
62
```

output

```
25
-12
77
```

Steps in C Compilation

- Really, generating an executable has more steps than you might expect.



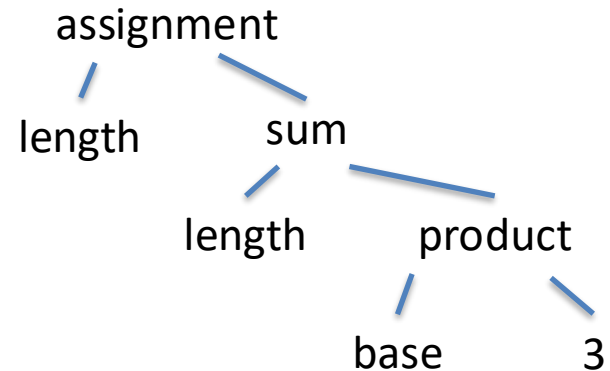
Steps in C Compilation

Tokens

```
length += base * 3 ;
```

Parsing

Parse Tree



Code Generation

Assembly Code

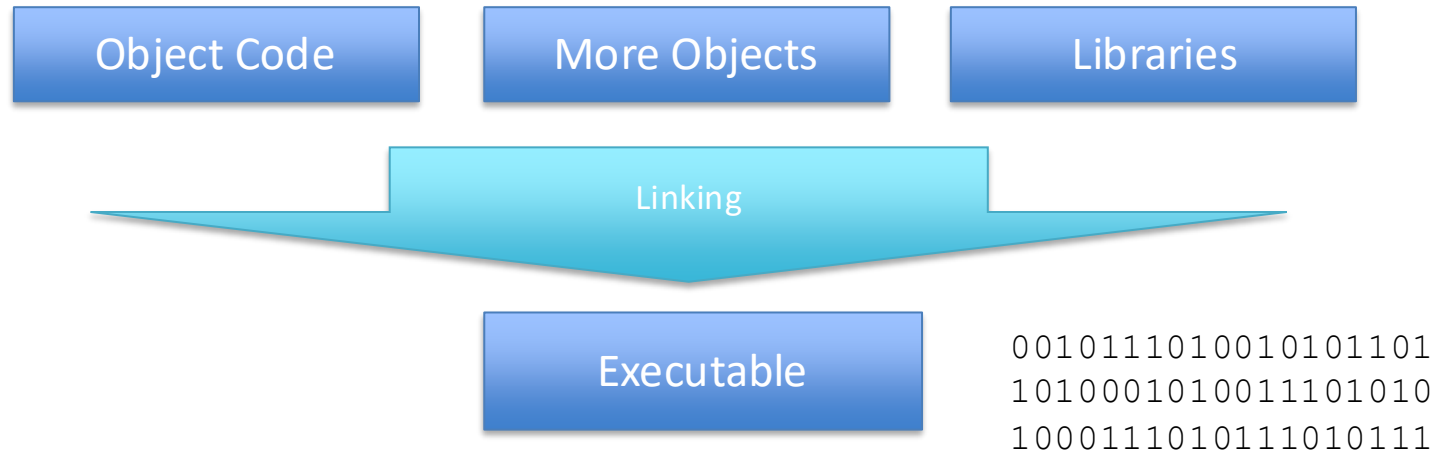
```
mov  base, %ebx
imul $3, %ebx
add  length, %ebx
mov  %ebx, length
```

Assembling

Object Code

```
0010111010010101101
```

Steps in C Compilation



The Preprocessor

- The preprocessor operates on the source code before the compiler even sees it.
- Performs basic text operations on the code
 - *Includes headers* : inserting code that enables use of code from other components
 - *Expands macros* : replacing macro names with corresponding definitions
 - More things we'll learn about later
- Lines starting with # are *preprocessor directives*
 - instructions processed (and removed) by the preprocessor

Preprocessor Constants

- Preprocessor macros give us a way to define named constants:

```
#define SIZE 25
```

Replace occurrences
of this ...

... with this.

Be careful, you probably don't
want a semi-colon here.

```
for ( int i = 0; i < SIZE; i++ )  
...
```



```
for ( int i = 0; i < 25; i++ )  
...
```

Looking for Tokens

- The compiler has to break the source into *tokens*
- This is called tokenization, scanning or *lexical analysis*
- A token can be:
 - An *identifier* (e.g., a variable or a function name)
 - A *keyword* (e.g., void or while)
 - A *literal value* (e.g., 3.1415, or “Hello World”)
 - An *operator* (e.g., *, ++ or >=)
 - An *explicit separator* (e.g., (, } or ;)
- White space and comments between the tokens are ignored

Identifiers

- *Identifiers* are variable, function or other names
- We get to choose names for these, but there are some rules:
 - An identifier consists of **letters**, **digits** or **underscores**
 - But, a variable can't start with a digit
 - So **x2** is an identifier
 - but **2x** isn't (it's a number followed by an identifier)
- Identifiers are case sensitive in C
 - So **myValue** is not the same as **myvalue**

Reserved Keywords

- Of course, some words in C already have special meaning
- In C89, you can't use these as identifiers
`auto`, `break`, `case`, `char`, `const`,
`continue`, `default`, `do`, `double`, `else`,
`enum`, `extern`, `float`, `for`, `goto`, `if`,
`int`, `long`, `register`, `return`, `short`,
`signed`, `sizeof`, `static`, `struct`,
`switch`, `typedef`, `union`, `unsigned`,
`void`, `volatile`, `while`
- C99 adds a few more:
`_Bool`, `_Complex`, `_Imaginary`, `inline`,
`_restrict`

Fun with Lexical Analysis

- What are the tokens in:

`a + ++b >= c-- -d`

- How about now:

`a+++b>=c---d`

- There are lots of things this could mean:

```
a ++ + b >= c - -- d   ?
```

```
a ++ + b > = c -- - d   ?
```

```
a + + + b > = c - - - d   ?
```

- This isn't really about precedence.
 - We can't even think about precedence until we know what the operators are.

The Scanner is Greedy

- The scanner works from left to right, grabbing the longest token it can
 - This is called *maximal munch*
- So, for our example:

a +++b>=c---d

(because a is a token, but a+ isn't)

a ++ +b>=c---d

(because ++ is a token, but +++ isn't)

a ++ + b>=c---d

(because + is a token, but +b isn't)

a ++ + b >=c---d

(because b is a token, but b> isn't)

a ++ + b >= c---d

(because >= is a token, but >=c isn't)

a ++ + b >= c ---d

(because c is a token, but c- isn't)

a ++ + b >= c -- -d

(because -- is a token, but ---isn't)

a ++ + b >= c -- - d

(because - is a token, but -d isn't)

Be the Scanner

- In the following expression, how many tokens are there?

```
---xy+=x++y2>>2z;
```

- What are they?
- This fragment of code wouldn't parse as a legal expression ...
- ... but the scanner would still be able to break it into individual tokens.

Coding Style Conventions

- There are lots of ways you **can** write a working program
- But there's a difference between what you can do and what you should do
- Consider this submission from the first International Obfuscated C Code Contest:

```
int i;main(){for(;i["]<i;++i){--i;}";read('-'-'-',i+++ "hell\
o, world!\n",'/'/'/'/'));}read(j,i,p){write(j/p+p,i---j,i/i);}
```

– I'm told it's a "Hello World" program

Coding Style Conventions

- Or this one from the 1993 contest:

```
O5 (O2,O7,O3)char**O7;{return!(O2+~O1+O1)?O0:!(O2==O2>O1)?printf("\045\157\012"
,O5(O12,O7+O1,O0)):!(O2==O2>>O1)?(*O7<=O67&&*O7>O57?O5(O3,O7,*(*O7)++-O6O+O1O
*O3):O3)!:!(O2
-==O3-
~O3)?
(O72>*
O7&&O6O
<==*O7
?O5(O4
,O7,O12
*O3-O6O
+*(*O7
)++) :O3
):!(O2
-==!O3+
!!O3)?(
**O7>O57
&&*O7
<=O71?
O5(O5,
O7,*(*
O7)+++
O3*O2O
-O6O):
**O7<=
O1O6&&
O01O1<=
**O7?O5
(O5,O7
,O2O*O3
+*(*O7)
++-O67)
:O14O<*
O7&&*
O7<O147
?O5(O5,
O7,-O127
+*(*O7
)+++O2O
*O3):O3
):!(
O2==O2-
O1)?(*
O7==O5O
?O5O**
++*O7,
O5(O13,
O7,O5(
O12,O7
,O0)):*
*O7<O56
&&O54<*
*O7?O55
***+*
O7,-O5(
O6,O7,
O0):O54
>*O7&&
O52<*
O7?O5O*
*(*O7)
++,O5(O6
,O7,O0
):!(**
O7^O17O
)||!(
O13O^**
O7)?*++
*O7,O5
(O5,O7
,O0):*
*O7==O144
||**O7
==O1O4
?++*O7
,O5(O4,
O7,O0):
O5(O3
,O7,O0
)):!--
O2?(*
*O7==O52
?O5(O7
,O7,O3*
(*++*O7
,O5(O6
,O7,O0)
)):!(
O45-*
O7)?O5(
O7,O7,
O3%(O3+(*O7)++,
O7^O57)?O5(O7,
O6,O7,O0))) :O3
):!(**
O7,O0)
)):!(**
O7,O7,
O3-*++
*O7,O5(
O6,O7,O0))) :O3
):!(O2
+==O1-O2
)?O5(O7
,O7,O5(O6,O7,
O0)):!(
O2+==O2/
O2)?(!(*
*O7-O53)?O5(O11,O7,O3+(++*O7,O5(O1O,O7,O0))) :!(O55^*O7)?O5(O11,O7,O3-(O3+(*O7
)++,O5(O01O,O7,O0))) :O3):!(O2==O563&O215)?O5(O11,O7,O5(O1O,O7,O0)): (++*O7,O3);}
```

Coding Style Conventions

- These examples are fun (citation needed) but deliberately hard to read and understand
- Normally, this is the opposite of what we want
- In CSC 230, we adopt some coding style conventions, rules for:
 - Naming conventions
 - Spacing and indentation
 - Where important comments go and what they contain
- Fortunately, editors can often help us with this.

CSC 230 Style Guidelines

- A Javadoc-style block comment at the top of each source file
 - With a `@file` tag giving the filename
 - And an `@author` tag with name and unity ID.
 - And a brief description of what the program does

```
/** I'm a program that reads in a list of words.  
    @file wordList.c  
    @author Bill Smith bsmith97  
    */
```

```
#include <stdio.h>  
#include <stdlib.h>
```

```
...
```

CSC 230 Style Guidelines

- A Javadoc-style block comment at the top of each function:
 - A sentence or two describing the function's purpose
 - @param tags for each parameter
 - @return tag for the return value

Omit if not needed.

```
/** Print a given number of copies of a given character.  
 * @param ch The character to print  
 * @param n The number of times to print ch.  
 */  
void printSymbol( char ch, int n )  
{  
    . . .
```

Extra stars are OK, if you like them.

CSC 230 Style Guidelines

- A good, Javadoc-style comment on each constant, global variable and type definition.
- Magic numbers, avoid bare constants for:
 - **Any value that could have an explanation**

```
area = radius * radius * 3.1415926;
```

- **Any potentially tunable parameter**

```
score += 350;
```

- **Any value that needs to occur multiple times**

```
for ( int i = 0; i <= 99; i++ )  
    ...;
```

```
for ( int j = 99; j >= 0; j-- )  
    ...;
```

CSC 230 Style Guidelines

- Curly Bracket Placement
 - For function definitions, the opening curly bracket goes on the next line (to make functions stand out)

```
int invert( double x, int y )  
{  
    ... body of function ...  
}
```

CSC 230 Style Guidelines

- Curly Bracket Placement
 - Other than function definitions, curly brackets go on the same line as the start of the statement.

```
for ( int i = 0; i < N; i++ ) {  
    ...  
}
```

```
if ( spiderSize == ITSY_BITS ) {  
    ...  
}
```

```
struct CoffeePot {  
    ...  
}
```

CSC 230 Style Guidelines

- Indentation
 - No hard tabs, just indent with spaces (why?)
 - Indent using any number of spaces you want, 2 spaces, 3 spaces ... maybe 4 spaces.
 - But, indentation should be consistent.
- Just one statement per line

CSC 230 Style Guidelines

- Global Variables
 - Avoid these. Only use them where the assignment says to.
 - In general, functions communicate with the rest of the program via parameters and return values.

```
#include <stdio.h>
#include <stdlib.h>

/** Any code can access me. */
float cost = 25.88;

void someFunction()
{
    . . .
}
```

Don't do this,
unless the design
says to.